EXPRESSIONS, FUNCTIONS, AND CONTROL Meta

COMPUTER SCIENCE MENTORS 61A

January 28, 2026 – January 30, 2026

Example Timeline

- Teaching tips:

  Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind. It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

  However, the time in brackets will add up to 50 minutes, and is intended to be a suggested outline for your section.

- Intros, Ice Breakers, & Logstics [10 min]

  Introduce yourself, and give an overview of what CSM offers. Start off with an icebreaker of your choice!

- Mini-lecture: Python Expressions, Functions, Control [10 min]
- Q1: Potpourri [5 min]

  Feel free to skip some parts!

- Q2: Funky Functions [7 min]
- Q3: Score Needed [7 min]
- Q4: Divisibility Check [5 min]
- Q5: Fizz Buzz [7 min]
- Questions [5 min]

  Save some time at the end to answer questions about this week's content or CSM!

1 Expressions

1. What Would Python Display?

```
>>> 3

3
>>> "cs61a"
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

```
'cs61a'
>>> x = 3
>>> x

3
>>> x = print("cs61a")
cs61a
>>> x


>>> print(print(print("cs61a")))


cs61a
None
None

>>> def f1(x):
...     return x + 1
>>> f1(3)

4
>>> f1(2) + f1(2 + 3)

9
>>> def f2(y):
...     return y / 0
>>> f2(4)

ZeroDivisionError: division by zero
>>> def f3(x, y):
...     if x > y:
...             return x
...     elif x == y:
...             return x + y
...     else:
...             return y
>>> f3(1, 2)

2
>>> f3(5, 5)

10
>>> 1 or 2 or 3

1
>>> 1 or 0 or 3
```

```
1
>>> 4 and (2 or 1/0)
2
>>> 0 or (not 1 and 3)
False
>>> (2 or 1/0) and (False or (True and (0 or 1)))
1
```

Suggested Time: 7 min

- You can probably skip a lot of these as soon as your students get the idea.
- Introduce the idea of booleans, or/and, and that print returns None.
- Your students might be new to solving "compound" questions like the last one; Consider giving them tips on how to break down such problems.

2 Functions

1. For the following expressions, simplify the operands in the order of evaluation of the entire expression

   Example: `add(3, mul(4, 5))`

   Order of Evaluation: `add(3, mul(4, 5))` → `add(3, 20)` → `23`

   (a) `add(1, mul(2, 3))`

   ```
   add(1, mul(2, 3))
   add(1, 6)
   7
   ```

   (b) `add(mul(2, 3), add(1, 4))`

   ```
   add(mul(2, 3), add(1, 4))
   add(6, add(1, 4))
   add(6, 5)
   11
   ```

   (c) `max(mul(1, 2), add(5, 6), 3, mul(mul(3, 4), 1), 7)`

   ```
   max(mul(1, 2), add(5, 6), 3, mul(mul(3, 4), 1), 7)
   max(2, add(5, 6), 3, mul(mul(3, 4), 1), 7)
   max(2, 11, 3, mul(mul(3, 4), 1), 7)
   max(2, 11, 3, mul(12, 1), 7)
   max(2, 11, 3, 12, 7)
   12
   ```

Suggested Time: 7 min

- Show students how to tackle problems with nested expressions (like the process in the solution for this problem).
- If students are comfortable with nested expressions, feel free to go over the example and skip straight to the last subpart.

2. Sally really wants to know what she needs to score to pass her class! Fill out the function `score_needed` that takes in $current\_grade$, her current grade, num_exams, the number of exams taken so far (not including her final exam), and $pass\_grade$, the grade needed to pass. Assume that every exam, including the final exam, has the same weight.

```
def score_needed(current_grade, num_exams, pass_grade):
    '''
    >>> score_needed(89, 4, 90)
    94
    >>> score_needed(65, 2, 70)
    80
    >>> score_needed(77, 10, 78)
    88
    '''
```

```
def score_needed(current_grade, num_exams, pass_grade):
    total_pts = current_grade * num_exams
    total_needed = pass_grade * (num_exams + 1)
    return total_needed - total_pts
```

Suggested Time: 10 min Make sure that students understand the question prompt and how to come up with the formula.

3 Control

1. Write a function that returns `True` if a number is divisible by 4, 1 if a number is divisible by 7 and is not already divisible by 4, and returns `False` if neither condition is fulfilled.

```
def divisibility_check(num):
```

```
def divisibility_check(num):
    if num % 4 == 0:
        return True
    elif num % 7 == 0:
        return 1
    else:
        return False
```

This also works as an alternate solution:

```
def divisibility_check(num):
    return True if num % 4 == 0 else 1 if num % 7 == 0 else False
```

Suggested Time: 5 min

2. Implement `fizzbuzz(n)`, which prints numbers from 1 to `n` (inclusive). However, for numbers divisible by 3, print "fizz". For numbers divisible by 5, print "buzz". For numbers divisible by both 3 and 5, print "fizzbuzz".

```
def fizzbuzz(n):
    """
    >>> result = fizzbuzz(16)
    1
    2
    fizz
    4
    buzz
    fizz
    7
    8
    fizz
    buzz
    11
    fizz
    13
    14
    fizzbuzz
    16
    >>> result is None
    True
    """


    i = 1
    while i <= n:
        if i % 3 == 0 and i % 5 == 0:
            print('fizzbuzz')
        elif i % 3 == 0:
            print('fizz')
        elif i % 5 == 0:
            print('buzz')
        else:
            print(i)
        i += 1
```

We must put the condition `i % 3 == 0 and i % 5 == 0` in the first **`if`**. For example, if we were to write the body of the while loop with the first two conditions switched.

```
        if i % 3 == 0:
            print('fizz')
        elif i % 3 == 0 and i % 5 == 0:
            print('fizzbuzz')
        elif i % 5 == 0:
            print('buzz')
        else:
            print(i)
```

then we may print out 'fizz' for a number like 15 which would be incorrect.

Suggested Time: 10 min